Hemlo

HEMLO · AUTHENTICATION

Hemlo ID: authentication system design

Status: built (phases 1, 3 and 4; WhatsApp and SMS not connected yet) · code in `platform/` · Date: 28 September 2026 · Scope: every Hemlo app (Systems, School, Studio portal, Tools accounts, mobile apps, internal admin)

حلولٌ بإتقان.

0. The decisions on one page

QUESTION	DECISION
Library	**Better Auth** (1.6 stable, 1.7 when released), self-hosted in our own database. No Clerk or Auth0.
Where it runs	A Cloudflare Worker (Hono) at `api.hemlohq.com/v1/auth/*`; the login screens at `account.hemlohq.com`
Database	Neon Postgres through Cloudflare Hyperdrive, with Drizzle schemas
How people sign in	**No passwords.** Phone + code over **WhatsApp** (SMS as fallback), email + code, **passkeys** (fingerprint / face), optional Google
One login everywhere	One session cookie on `.hemlohq.com`, so signing in once works on every subdomain
Businesses and staff	Organizations, members, roles and locations (Better Auth organization plugin, with teams used as locations)
Shared shop devices	**Counter mode:** the owner trusts a device once, then staff switch on it with a personal PIN
Customers of our customers	No passwords, no accounts to manage: signed links, plus phone codes for portals (parents, repair status)
Mobile apps and custom domains	Hemlo ID acts as an OAuth 2.1 / OpenID provider (code + PKCE)
Who can do what	Permissions checked in the API (`resource:action`), with Postgres row-level security as a second lock
Cost at launch	$0. After launch, about 13 IQD per WhatsApp code and 80–140 IQD per SMS code

1. What "perfect" means for Hemlo

The system must fit how Iraqi businesses actually work:

1. **Phone first.** Many owners and nearly all staff have a phone number and WhatsApp, not a work email.
2. **Shared devices.** One phone or tablet sits at the counter, and three people use it in a day.
3. **Weak internet and power cuts.** Signing in needs the internet, but a signed-in counter device must keep working offline.
4. **Arabic, Kurdish and English**, right to left, and digits typed as `٠٧٧٠` or `0770`.
5. **Recycled numbers.** Iraqi carriers reissue old numbers, so a phone number alone is not a lifetime identity.
6. **$0 until paying customers**, and costs that stay small per user afterwards.
7. **One login for every Hemlo product,** so a customer of Systems can add School or a Studio portal in two clicks.

And the usual security standard: nothing guessable, nothing stored in plain text, every sign-in rate limited and logged.

2. Who signs in

IDENTITY	WHO	HOW THEY SIGN IN	WHERE THEIR POWER ENDS
Hemlo ID (a person)	Owners, managers, staff, accountants, parents	Phone code, email code, passkey, Google	Only what their memberships allow
Organization (a business)	A shop, a school, a company	Not a login: it's what members belong to	Its own data only
Counter device	The shop's shared phone or tablet	Trusted once by an owner; staff unlock with a PIN	One organization, one location, counter permissions
Portal visitor	A repair customer, a parent	Signed link, or phone code for portals	Only the records linked to their phone number
API key	An integration built by Studio	Secret key in a header	Scopes chosen when the key is made
Hemlo staff (us)	Admin panel	Passkey only, behind Cloudflare Access	Admin actions, all logged

Portal visitors are ordinary Hemlo IDs with **no organization membership**. A parent who later opens their own shop already has an account.

3. How people sign in

3.1 The methods

METHOD	USED FOR	WHY	COST
Phone + WhatsApp code	Default for everyone	Everyone has WhatsApp; the code arrives in a chat they already trust	≈ $0.009 (≈ 13 IQD) per code (Meta, Iraq rate)
Phone + SMS code	Fallback after 30 seconds, or when WhatsApp isn't installed	Works on any phone	80–140 IQD per code (Iraqi gateways)
Email + code	People who prefer email; stage 1 launch	Free	$0 (Resend free tier)
Passkey (fingerprint, face, screen lock)	Offered after the first sign-in; required for Hemlo staff	Fastest, free, can't be phished	$0
Google	Optional button	Many Android users are already signed in to Google	$0
Password	**Not offered**	Reused, forgotten, phished, and slow to hash on the Worker's CPU limit	n/a

A **passkey** created on `hemlohq.com` works on every Hemlo subdomain, so after the first sign-in most people never type a code again.

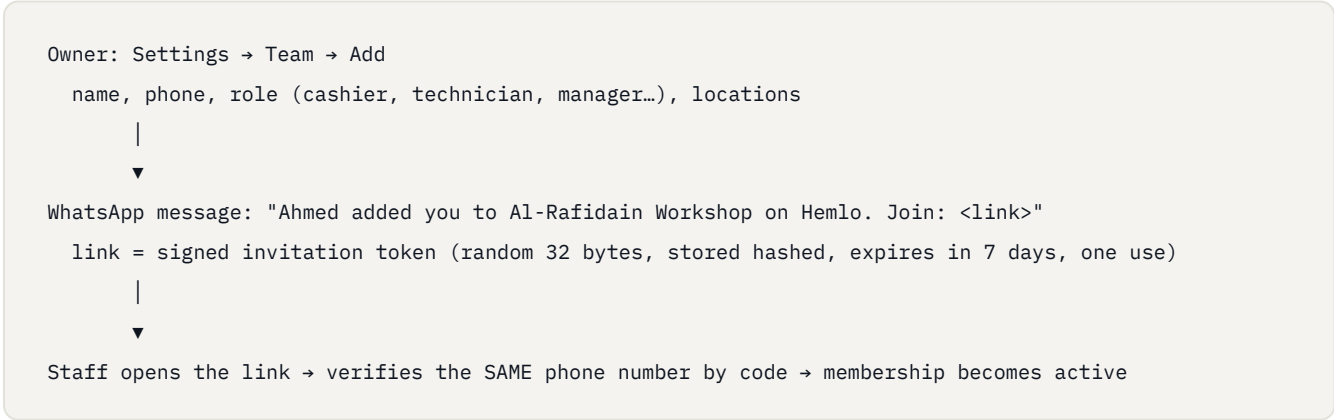
When the iOS app ships with Google sign-in, Apple requires **Sign in with Apple** as well. Add it then.

3.2 Sign in and sign up are the same screen



- **Codes:** 6 digits, valid 5 minutes, 5 tries, then the code is dead. They're stored as a keyed hash (HMAC-SHA256 with a server secret), never in plain text.
- **Resend:** waits of 30 s, 60 s and then 2 minutes, with at most 5 codes per number per hour.
- **Enumeration:** the reply never reveals whether a number or email is registered.

3.3 Owner invites staff



The phone code proves the invite reached the right person. A forwarded link is useless without that phone.

3.4 Counter mode (shared devices)

This is the feature most Iraqi shops need, and most systems get wrong.

```
1. Owner signs in on the shop tablet → "Use this device as a counter"
   → a DEVICE session is created: bound to one organization + one location,
     counter permissions only, lasts 90 days, can be revoked from the owner's phone.
2. Each staff member sets a 4–6 digit PIN once (on their own phone, while signed in).
3. At the counter: tap your name → enter your PIN → you're working as yourself.
4. After 10 minutes idle, or when "Lock" is tapped, the screen returns to the name list.
```

- Every sale, job and refund is recorded under **the person**, not "the tablet".
- **PIN safety** comes from the device binding and lockout (5 wrong PINs locks that person on that device, and the owner is notified). The PIN is stored as an HMAC with a server secret. It's never usable on its own, and it only works on a trusted device.
- **Sensitive actions** always ask for more (see 3.6): refunds above a limit, deleting records, changing prices, reports with totals, team and billing settings.
- **Offline:** the device session keeps the counter working without internet. Sales queue locally and sync when the connection returns. Revocations take effect at the next sync.

3.5 One login across Hemlo

- The login screens live on `account.hemlohq.com`. Any app sends people there with `?return_to=https://systems.hemlohq.com/app` (the address must be on an allowlist).
- The session cookie is set for `Domain=.hemlohq.com`, so `systems.`, `school.`, `studio.` and `tools.` all see it.
- Apps call `api.hemlohq.com` with `credentials: "include"`. Because it's the same site, the `SameSite=Lax` cookie is sent, and CORS only allows `*.hemlohq.com` origins.
- **Custom domains** (a Studio client's app on their own domain, or a white-label edition) can't share the cookie. They sign in through **OpenID Connect** instead, redirecting to `account.hemlohq.com`. Passkeys therefore still work, because the passkey belongs to `hemlohq.com`.
- **Mobile apps** use the same OpenID flow (authorization code + PKCE) and keep the token in the Keychain or Keystore, never in plain storage.

3.6 Step-up: proving it's you again

A session counts as **fresh** for 10 minutes after a sign-in. Sensitive actions need a fresh session, and if it isn't fresh the user confirms with a passkey or a new code. These actions need it:

- changing the phone number or email
- adding or removing owners and admins
- billing and plan changes
- deleting an organization, or exporting all data
- creating API keys
- turning a device into a counter

3.7 Recovery and recycled numbers

SITUATION	WHAT HAPPENS
Lost phone, same number	Get a new SIM for the same number and sign in with a code. Old devices can be signed out from the new one.
Lost number	Sign in with email or a passkey, then change the number (step-up required)
Owner with only a phone number	Onboarding asks them to add a passkey or email within 7 days: "Add a backup way to sign in"
Recycled number: someone new gets an old owner's number	Two protections: (1) a code alone can't open an owner or admin account that has a passkey or email; it asks for that second method. (2) A number unused for 6+ months, signing in on a new device, must confirm a backup method before it can see any organization.
Nothing left at all	Support-assisted recovery: business documents, a 72-hour delay, and notices to every old channel during the delay

3.8 Sessions people can see

Account → **Devices** lists every session: device, browser, approximate city, and when it was last active. Each one can be signed out individually, or all at once.

A **new-device alert** goes to WhatsApp (or email) for owners and admins: "New sign-in on Chrome, Baghdad, 14:02. Not you? Tap to sign out."

4. Sessions and tokens

ITEM	DESIGN
Session token	Random and opaque (Better Auth database sessions); the cookie is signed with the server secret
Cookie	`__Secure-hemlo.session_token` : `HttpOnly; Secure; SameSite=Lax; Domain=.hemlohq.com; Path=/`
Fast checks	Better Auth **cookie cache**: a signed, short-lived (5 min) cookie holding the session summary, so most requests skip the database. Security-critical routes always read the database.
Lifetimes	Owners and staff: 30 days, extended daily while in use. Counter devices: 90 days, with a 10-minute idle lock for the person. Hemlo staff: 12 hours.
Revocation	Immediate in the database; at most 5 minutes behind on routes that trust the cookie cache
Mobile	Bearer token from the OpenID flow, rotated refresh tokens, stored in the OS keystore
CSRF	SameSite=Lax + an Origin/Referer check against trusted origins + state changes only via POST, PUT or DELETE

ITEM	DESIGN
Later	Better Auth 1.7's **public-key session verification** lets any Worker check a session without a database call. Adopt it when it's stable.

5. Authorization: who can do what

5.1 Three questions on every request

```
request ──► 1. Who is it?        session → user            (401 if none)
        ──► 2. Which business?   active organization → membership, status active  (403)
        ──► 3. Allowed?          role → permissions includes "sales:refund"?        (403)
                                  + plan includes this edition? + location allowed?
```

5.2 Roles and permissions

Permissions are `resource:action` pairs, defined once in `packages/auth/permissions.ts` using Better Auth's access-control helpers. Each edition adds its own resources.

ROLE	CORE PERMISSIONS	WORKSHOPS EDITION ADDS
Owner	everything, billing, delete organization	everything
Admin	team, settings, all data, no billing	everything
Manager	sales, refunds, stock, reports for their locations	jobs, warranty approvals
Cashier	sales:create, customers:read/create	jobs:read
Technician	customers:read	jobs:update-own, parts:use
Accountant	reports:read, sales:read, export	none
Viewer	read only	read only

Owners can make **custom roles** later (a stored role with a chosen permission list).

5.3 Locations

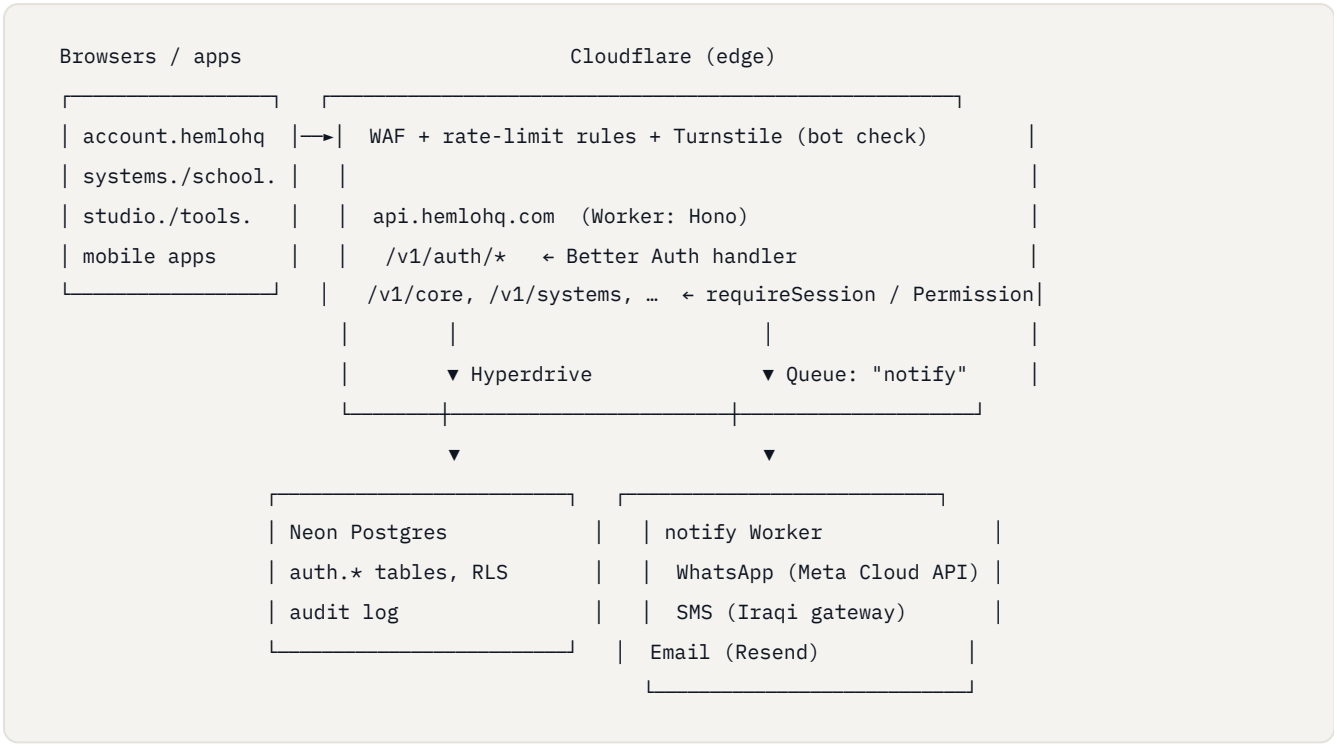
Better Auth **teams** are used as **locations** (branches of a shop). A member can be limited to specific locations, and counter devices are bound to exactly one.

5.4 Two locks, not one

1. **API check:** every route declares its permission: `requirePermission("sales:refund")`.
2. **Database check:** each request runs inside a transaction with `SET LOCAL app.org_id = …`. Postgres **row-level security** policies only return rows of that organization. A forgotten check in code still can't leak another shop's data.

Plans and entitlements (which editions an organization pays for) are checked at the same gate, but they live in the billing module, not in auth.

6. Architecture



- **Codes are sent in the background** (`ctx.waitUntil`), so the "send code" request answers at once. A slow WhatsApp API never makes the screen hang, and failures fall back to SMS. Move this to a Cloudflare Queue when retries are needed.
- **`account.hemlohq.com`** is a small Astro site (the same stack as the website): a few fast pages that work on weak connections, with the interactive parts as small islands.

6.1 Code layout (in the monorepo)

```
packages/
  auth/
    server.ts         betterAuth({...}) configuration
    permissions.ts    resources, actions, roles (shared by server and UI)
    phone.ts          normalise Iraqi numbers (+964, ٩-٠ digits, leading 0)
    middleware.ts     requireSession, requireOrg, requirePermission, requireFresh
    counter.ts        device sessions + PIN switch
    client.ts         createAuthClient for every app
  notify/             sendOtp(channel, to, code, locale) → WhatsApp | SMS | email
apps/
  api/                mounts auth.handler at /v1/auth/*
  account/            sign-in, sign-up, devices, security, organizations screens
```

6.2 Configuration sketch

This is illustrative. Check option names against the Better Auth docs when building.

```
export const auth = betterAuth({
  appName: "Hemlo",
  baseURL: "https://api.hemlohq.com",
  basePath: "/v1/auth",
  database: drizzleAdapter(db, { provider: "pg" }),
  trustedOrigins: ["https://hemlohq.com", "https://*.hemlohq.com"],
  emailAndPassword: { enabled: false },                    // no passwords
  session: {
    expiresIn: 60 * 60 * 24 * 30, updateAge: 60 * 60 * 24,
    freshAge: 60 * 10,
    cookieCache: { enabled: true, maxAge: 60 * 5 },
  },
  advanced: {
    cookiePrefix: "hemlo", useSecureCookies: true,
    crossSubDomainCookies: { enabled: true, domain: ".hemlohq.com" },
  },
  rateLimit: { enabled: true, storage: "database",
    customRules: { "/phone-number/send-otp": { window: 3600, max: 5 },
                   "/email-otp/send-verification-otp": { window: 3600, max: 5 } } },
  plugins: [
    phoneNumber({ otpLength: 6, expiresIn: 300, allowedAttempts: 5,
      sendOTP: ({ phoneNumber, code }, ctx) => notify.enqueueOtp({ to: phoneNumber, code, channel: "whatsap
    emailOTP({ sendVerificationOTP: ({ email, otp }) => notify.enqueueOtp({ to: email, code: otp, channel:
    passkey({ rpID: "hemlohq.com", rpName: "Hemlo" }),
    organization({ ac, roles, teams: { enabled: true } }),
    oidcProvider({ loginPage: "https://account.hemlohq.com/sign-in" }), // mobile + custom domains
    apiKey(), admin(), lastLoginMethod(),
  ],
  databaseHooks: { session: { create: { after: auditSignIn } } },
});
```

7. Data model

Better Auth creates the core tables. We add a few columns and six tables of our own (`auth_event` , `known_device` , `staff_invite` , `counter_device` , `counter_pin` , `counter_lock`).

TABLE	KEY COLUMNS	NOTES
`user`	id, name, email?, email_verified, **phone_number?**, phone_verified, **locale** (ar/ckb/en), image	At least one of phone or email is required
`account`	user_id, provider (google, phone, email…), provider_account_id	One person, many sign-in methods
`session`	token (hashed), user_id, expires_at, ip, user_agent, **active_organization_id**, **kind** (web / mobile / counter)	
`verification`	identifier, value (hashed code), expires_at, attempts	Codes and invitation tokens
`passkey`	user_id, credential_id, public_key, counter, device_type, name	"Ahmed's iPhone"
`organization`	id, name, slug, logo, **country**, **default_locale**, created_at	
`member`	organization_id, user_id, role, **status** (invited / active / suspended), **pin_hash?**	PIN only if counter mode is on
`team` → used as **location**	organization_id, name, address	
`team_member`	team_id, user_id	Which locations a member works in
`invitation`	organization_id, phone or email, role, token (hashed), expires_at, inviter_id	
`counter_device`	id, organization_id, team_id, name, session_id, trusted_by, last_seen, revoked_at	Ours
`apikey`	organization_id, hashed key, prefix, scopes, last_used	Shown once at creation
`auth_event`	at, user_id?, org_id?, kind (sign_in, failed_code, new_device, role_change…), ip (truncated after 30 days), user_agent, detail	Ours; append-only audit log

8. Sending codes

STEP	RULE
Channel order	WhatsApp → SMS (after 30 s, or if WhatsApp reports "not on WhatsApp") → email if one exists
WhatsApp	Meta **Cloud API** directly (no middleman), using an **authentication template** with a "Copy code" button, in Arabic, Kurdish and English. Meta allows about 250 new recipients a day until the business is verified, so verify the Meta business early.
SMS	An Iraqi gateway with direct carrier routes (e.g. OTPIQ or Bulk SMS Iraq). Keep it behind the `notify` interface so it can be swapped.
Email	Resend (the free tier covers launch), from `no-reply@hemlohq.com`, with SPF, DKIM and DMARC set in Cloudflare
Fraud (SMS pumping)	Only +964 numbers by default. Caps per number, per IP and per number prefix. Turnstile appears when a pattern looks automated, and there's an alert when daily spend passes a limit.
Message	"رمز الدخول إلى هيملو: 482913. لا تشاركه مع أحد." / "Your Hemlo code: 482913. Don't share it." The code comes first, so it shows in notifications.

9. Security checklist

- [ ] No passwords stored anywhere. Codes, invitation tokens, PINs and API keys are all stored hashed.
- [ ] Rate limits in three layers: Cloudflare rules, the Better Auth limiter, and per-number caps.
- [ ] The same reply whether an account exists or not.
- [ ] Turnstile on "send code" when the risk is high, and on sign-up.
- [ ] Step-up (a fresh session) for sensitive actions.
- [ ] New-device alerts for owners and admins; a device list with sign-out.
- [ ] Protection against recycled numbers (3.7).
- [ ] Row-level security in Postgres as the second lock.
- [ ] `return_to` allowlist (no open redirects); strict CORS; HSTS and CSP on `account.`.
- [ ] Hemlo staff: passkeys only, `admin.` behind Cloudflare Access, every action in `auth_event`.
- [ ] Secrets in Cloudflare secrets, rotated yearly or on any staff change.
- [ ] Privacy: IPs truncated after 30 days; users can export or delete their account; nothing sold or shared.

10. Iraqi details that make it feel right

- The phone field accepts `0770 123 4567`, `770-123-4567`, `+964 770…` and Arabic-Indic digits `٠٧٧٠…`, and always stores `+9647701234567`.
- Code boxes stay **left to right** inside Arabic and Kurdish pages, with `inputmode="numeric"` and `autocomplete="one-time-code"` so phones fill the code in automatically.
- An **"Open WhatsApp"** button sits next to the code field.
- Every message, email and error is written in all three languages, and the language follows the person, not the device.
- Screens are small and fast: the sign-in page stays under about 50 KB so it loads on a weak signal.
- Counter mode shows a name list with photos or initials. Tapping is faster than typing at a busy counter.

11. Build plan

PHASE	WHAT SHIPS	COST
1. Foundation (≈ 1–2 weeks)	Email code + passkeys, sessions across subdomains, organizations, roles, locations, permission middleware, RLS, audit log, rate limits, account screens (sign-in, devices, organizations), in ar/ckb/en	$0
2. Phone	WhatsApp codes (Meta business verified), SMS fallback, staff invitations on WhatsApp, new-device alerts	Per code only
3. Counter mode	Trusted devices, PIN switch, idle lock, offline device sessions (ships with the Systems Workshops edition)	$0
4. Beyond the browser	OpenID provider for mobile apps and custom domains, API keys, Hemlo admin panel	$0
5. Enterprise (only if asked)	SSO with a company's own identity provider, SCIM user sync	$0

Tests before each phase ships:

- Unit tests: phone normalisation, permissions.
- Integration tests: every flow in section 3, including wrong codes, expired links and revoked devices.
- End-to-end browser tests: sign-in on a phone-sized screen in Arabic.
- A spend alert on messaging.

12. Running costs

ITEM	AT LAUNCH	1,000 ACTIVE USERS
Worker, Hyperdrive, Turnstile, Queues	$0	≈ $5/month (Workers paid plan, for CPU headroom)
Neon Postgres	$0	$0–19/month
Email codes	$0	$0
WhatsApp codes (≈ 13 IQD each)	$0	≈ 3,000 codes → ≈ $27/month, and less as passkeys take over
SMS fallback (≈ 100 IQD each)	$0	a few hundred → ≈ $20/month

Passkeys are the cost control: every person who adds one stops needing paid codes.

13. Sources

- Better Auth 1.6 release: https://better-auth.com/blog/1-6
- Better Auth 1.7 release candidate: https://github.com/better-auth/better-auth/discussions/10250
- Better Auth plugins: https://better-auth.com/docs/plugins/phone-number · https://better-auth.com/docs/plugins/organization · https://better-auth.com/docs/plugins/passkey
- WhatsApp pricing: https://developers.facebook.com/documentation/business-messaging/whatsapp/pricing · https://www.ycloud.com/blog/whatsapp-api-message-pricing-update-effective-october-1-2026
- Iraqi OTP gateways: https://github.com/otpiq · https://bulksmsiraq.com/otp-api-in-iraq-iq-by-bulk-sms-iraq/

14. As built (28 September 2026)

The code is in `platform/` (see its README). It has 60 API tests on a real Postgres and a browser test of the screens in headless Edge. The browser test uses a virtual fingerprint to exercise passkeys.

Where the build differs from the plan above, and why:

TOPIC	WHAT WAS BUILT
Library	Better Auth **1.7.6** (stable). Passkeys, API keys and the OAuth provider come from its separate `@better-auth/*` packages.
Phone codes	Better Auth stores phone codes in plain text, so a database hook replaces each one with HMAC(code) before it is saved, and Hemlo checks codes itself (5 tries). Email codes use Better Auth's built-in hashing.

TOPIC	WHAT WAS BUILT
People with only a phone	Better Auth needs an email on every user. Phone sign-ups get a placeholder (`p964…@phone.hemlohq.invalid`) that is never shown and can never receive mail.
Invitations	Hemlo's own (`staff_invite`), because Better Auth's only take email addresses. They work by phone or email. A phone invitation returns a free `wa.me` link the owner sends from their own WhatsApp. Accepting needs the same verified phone or email, and the invitee's name comes from the invitation.
Second factor	A phone-only sign-in gets a session marked "second factor pending" when either: the account is an owner or admin with a passkey or verified email, or the account has been dormant for 180+ days. That session can do nothing except finish signing in with the passkey or email, which replaces it. It also can't change the phone number or add a passkey.
Counter mode	Trusting a device **ends the owner's session on it** (a clean handover). A signed-in person always acts as themselves. Counter screens send `x-hemlo-as: counter` to act as the device's unlocked staff member.
New sessions	Reopen the business the person used last.
Time	Better Auth's columns are `timestamp` without a zone, so every connection and process is forced to UTC. A 3-hour Baghdad offset was caught by the tests.
Authorization reads	Permission checks always read the session from the database. The 5-minute cookie cache is used only where being stale is harmless.
Tables	All tables live in the `public` schema. Row-level security is FORCEd on every business table, and the tests prove it using an ordinary database role.
Not connected yet	WhatsApp and SMS delivery. Phone sign-in stays switched off in production (`PHONE_SIGN_IN=0`) until one is connected in `packages/notify` .